Unity QuadSim Architecture Overview

Codebase structure, runtime flow, and principal source files

1. Purpose and Scope

Unity QuadSim is a Unity package for deterministic quadrotor simulation. The repository contains the runtime simulation code, configurable vehicle models, Unity prefabs and scene settings, local UI and camera tooling, external communication support, reinforcement-learning features, examples, tests, and release documentation.

The architecture separates simulation-wide responsibilities from per-drone responsibilities. SimCore owns time, stepping, global authority, scene configuration, and external transport. DroneCore owns each vehicle, including its command path, flight modes, controller, motor allocation, physical actuation, camera, and per-drone API. RobotCore and MathUtil provide sensing, georeferencing, noise, and coordinate-frame conversions shared by the vehicle layer.

This overview identifies the main subsystems and entry points rather than cataloging every file. The complete source listing is maintained separately for line-by-line public-release review.

Architecture at a Glance

```
Scene configuration -> QuadSimEnvironmentBootstrap -> SimulationManager and
DroneManager
Command source -> authority check -> FlightCommandProxy -> ModeCoordinator
Goal -> CascadedController -> ControlAllocator -> QuadActuationModel -> Unity Rigidbody
Unity physics state -> SensorManager and camera sensors -> APIs -> UI and RPC telemetry
```

Primary Design Characteristics

- Deterministic fixed-step execution is controlled explicitly rather than relying on Unity's normal FixedUpdate scheduling.
- The same global and per-drone APIs are used by internal scripts, the runtime UI, and external RPC handlers.
- Vehicle mass, rotor geometry, motor characteristics, camera settings, limits, and PID gains are loaded from YAML profiles.
- Coordinate conversions are centralized so Unity coordinates and exposed FLU, FRD, and ENU data remain consistent.

2. Simulation Kernel and Scene Startup

SimulationManager is the runtime backbone. In Awake it switches Unity physics to script-driven simulation and initializes ClockFactory at the configured base rate. During Update it converts free-running wall time or queued manual requests into a number of fixed simulation steps. Each step advances the simulation clock, invokes every registered ISimulatable in deterministic order, runs Physics.Simulate once, and then invokes the post-physics stage.

The three-part step contract is important to the rest of the project. PrePhysicsStep is where commands are consumed and actuator targets are produced. Physics.Simulate advances rigid bodies using one authoritative time increment. PostPhysicsStep is where state-dependent work such as sensor sampling and telemetry preparation occurs. Manual StepImmediate calls use the same path, so lockstep and reinforcement-learning clients do not bypass the normal vehicle lifecycle.

QuadSimEnvironmentBootstrap connects Unity scene configuration to the runtime. Before spawning and socket binding occur, it applies the selected drone profile, spawn origin and policy, simulation run mode, RPC address and ports, sensor seed and noise setting, geodetic origin, output frame, initial flight mode, payload selection, and wind configuration. QuadSimSceneConfig holds these values as a maintained ScriptableObject rather than distributing them across unrelated components.

SimCoreApi is the public facade for global operations. It delegates pause, resume, time scale, step, reset, and command-authority operations to SimulationManager and ControlAuthorityManager. SimCore intentionally does not reach into DroneCore; per-drone work goes through QuadSimApi. This keeps the package dependency direction clear and makes external integrations less dependent on scene component details.

Main Entry Files

- SimulationManager.cs - authoritative clock ownership, fixed-step scheduling, and physics execution.
- SimClock.cs and ClockFactory.cs - simulation time representation and scaled clock setup.
- StepContract.cs - ISimulatable lifecycle and deterministic ordering contract.
- QuadSimEnvironmentBootstrap.cs and QuadSimSceneConfig.cs - scene configuration application and storage.
- SimCoreApi.cs and ControlAuthorityManager.cs - global control facade and active command-source ownership.

3. Drone Lifecycle and Command Processing

DroneManager owns the active drone collection. It selects a YAML profile, resolves the matching prefab through DroneCatalog, spawns or reloads drones, assigns stable indexes and identities, tracks the selected drone, and forwards simulation start, pre-physics, and post-physics callbacks. Scene-level settings are passed into the drone before its sensors start so the initial sample already uses the correct frame, seed, and geographic origin.

QuadPawn is the central per-vehicle object. During initialization it resolves its Rigidbody and required components, loads and applies the selected DroneConfig, computes rotor-derived values, configures physical mass and inertia, initializes sensing and control, and exposes the completed subsystem set through QuadSimApi. During the simulation loop it coordinates command evaluation, control updates, actuation, collision state, sensor updates, and reset behavior.

Commands can originate from the built-in UI, internal scripts, or an external client. ControlAuthorityManager selects which source currently owns control at the simulation level. FlightCommandProxy is the per-drone gate: it accepts commands from the authorized source and presents the active goal to the controller through ICommandSource. This prevents the UI and an external process from silently driving the same drone at the same time.

ModeCoordinator tracks the active GoalMode and ControllerKind. Position, velocity, acceleration, angle, body-rate, passthrough, and wrench modes determine how the four command axes are interpreted. Mode or controller transitions trigger defined reset behavior so stale goals, integrators, or plant state do not leak into a new control mode.

Command Path

1. A source requests global authority through SimCoreApi.
2. The source selects a drone through DroneManager and QuadSimApi.
3. QuadSimApi forwards the command to FlightCommandProxy using the current mode.
4. ModeCoordinator determines the interpretation and transition/reset policy.
5. QuadPawn supplies the accepted goal and current sensor state to the controller during PrePhysicsStep.

Main Entry Files

- DroneManager.cs - drone registration, spawning, selection, reload, reset, and step dispatch.
- QuadPawn.cs and DroneRootBootstrap.cs - per-drone composition, configuration, and lifecycle setup.
- QuadSimApi.cs - stable per-drone command, state, sensor, controller, and reset facade.
- FlightCommandProxy.cs, ICommandSource.cs, and ModeCoordinator.cs - command gating and mode coordination.

4. Flight Control, Allocation, and Physical Actuation

CascadedController implements the primary flight-control path. It builds position, velocity, angle, and rate stages from the loaded gains and limits. Depending on the active mode, it begins at the appropriate stage rather than always running the full cascade. The result is a desired collective force and body torque, or direct motor-level output for supported bypass paths.

The axis controllers use QuadPIDController instances and share sensor data supplied by SensorManager. Position goals produce velocity setpoints; velocity control produces desired acceleration or specific force; angle control produces body-rate targets; and rate control produces torque demand. The controller retains telemetry snapshots and PID mappings for diagnostics and runtime tuning.

ControlAllocator converts requested roll, pitch, yaw, and collective thrust into normalized motor commands. It constructs an effectiveness matrix from the configured rotor positions, thrust directions, spin directions, and rotor physics. A pseudo-inverse provides the base allocation, followed by saturation and yaw-limiting behavior. Allocation diagnostics expose achieved demand, residual, scaling, rank, condition number, and saturated motors.

QuadActuationModel converts normalized motor commands into physical rotor thrust and reaction torque. It can model separate motor spin-up and spin-down time constants before applying force and torque to the drone Rigidbody. Keeping the controller, allocator, and plant model separate allows control behavior to be evaluated independently from motor response and physical application.

Supporting Files

- CascadedController.cs - control-stage coordination, setpoint progression, allocation, and controller telemetry.
- PositionController.cs, VelocityController.cs, AngleController.cs, and AcroController.cs - individual cascade stages.
- QuadPIDController.cs - reusable PID implementation used by the flight-control stages.
- ControlAllocator.cs - effectiveness matrix, pseudo-inverse allocation, saturation handling, and diagnostics.
- Quadactuationmodel.cs - motor lag, thrust/torque calculation, and Rigidbody force application.
- RotorMath.cs and Frames.cs - rotor-derived values and frame-aware vector/rotation conversions.

5. Sensors, Frames, and Vehicle Configuration

SensorManager owns the simulated sensor suite and its output frame. It initializes the sensors from the selected DroneConfig and scene environment, establishes deterministic noise streams from the master seed, and updates sensor outputs after physics. RobotCore and SensorData provide the common state representation consumed by controllers, APIs, telemetry, and tests.

IMUSensor derives accelerometer and gyroscope measurements from Rigidbody motion. GPSSensor reports position and velocity using GeoReferencingSystem, which converts between a local ENU tangent frame and latitude, longitude, and ellipsoidal height. MagSensor combines attitude, geographic location, and the maintained geomagnetic tables to produce magnetic-field measurements. BaroSensor derives pressure and altitude-related output. GaussianRng supports repeatable sensor noise.

Frames centralizes transformations between Unity and the exposed simulation conventions. Linear vectors, angular vectors, positions, and quaternions do not all transform identically, so keeping those conversions in one utility reduces sign and axis errors. The same frame selection is used when building the rotor effectiveness matrix and when packaging sensor, RPC, and reinforcement-learning state.

DroneConfigLoader first checks the host project's StreamingAssets configuration directory for an override and otherwise loads the packaged YAML profile from Resources. YamlParser maps the YAML into DroneConfigStructs covering the model, body, rotor instances, camera, flight limits, and controller gains. This makes new or modified vehicles primarily a data-authoring task rather than a controller rewrite.

Main Entry Files

- SensorManager.cs, SensorData.cs, and RobotCore.cs - sensor ownership and shared state.
- IMUSensor.cs, GPSSensor.cs, MagSensor.cs, and BaroSensor.cs - individual sensor models.
- GeoReferencingSystem.cs, GeoMagDeclination.cs, GeoMagneticTables.cs, and GaussianRng.cs - geographic, magnetic, and noise support.
- Frames.cs - coordinate and orientation conversion rules.
- DroneConfigLoader.cs, YamlParser.cs, and DroneConfigStructs.cs - configuration discovery, parsing, and data structures.
- Runtime/Resources/QuadSim/Configs/Drones - packaged Crazyflie, Flightmare, Sentinel, and standard quad profiles.

6. APIs, RPC, UI, Cameras, and Learning Workflows

QuadSim exposes two API levels. SimCoreApi addresses the simulation as a whole, while QuadSimApi addresses one registered drone. QuadSimApi provides mode and controller selection, command submission, sensor and telemetry retrieval, and targeted reset operations without requiring callers to manipulate the pawn's components directly.

ExternalRpcAdapter is the Unity-side integration point for external clients. RpcServerThread owns the ZeroMQ request/reply and publish sockets away from the Unity main thread. Incoming requests are queued to ExternalRpcAdapter, which drains them during Update, dispatches handlers against SimCoreApi or QuadSimApi, and returns MessagePack responses. The adapter also manages connection state, heartbeat timeout, subscriptions, telemetry publication, image/state/event sequences, and reinforcement-learning operations. This queue boundary is necessary because Unity objects must be accessed on the main thread.

The runtime ImGui layer uses the same API surface for local operation. ImGuiRuntimeBootstrap resolves the simulation and selected drone, drains deferred UI actions outside the rendering callback, binds the UI to the active camera, streams UI-owned commands when authority permits, and composes the taskbar, control panel, state HUD, PID settings, joystick view, and connection-approval popup.

CameraDirector and DroneCameraRig manage the live view camera and drone-relative mounts. DroneImageSensor captures images for external consumers without changing the flight-control path. Debug drawing and rotor geometry gizmos provide engineering visualization without becoming dependencies of the controller or physics code.

The RL layer captures normalized multi-agent state through RLAgentState, records collision and episode events, and uses ScenarioObjectManager to create and pool goals or obstacles described by external requests. These features sit above the same drone, stepping, sensor, and RPC layers used by ordinary simulation clients.

Main Entry Files

- ExternalRpcAdapter.cs, RpcServerThread.cs, and RpcProtocol.cs - transport, main-thread dispatch, and wire messages.
- ImGuiRuntimeBootstrap.cs and the RuntimeUI/ImGui panel files - local control and diagnostics.
- CameraDirector.cs, DroneCameraRig.cs, and DroneImageSensor.cs - view selection, mounts, and image capture.
- RLAgentState.cs, RLSceneObject.cs, and ScenarioObjectManager.cs - RL state and scenario objects.

7. Directory Map

The following directory map identifies where a reviewer should look for each major responsibility. It intentionally groups supporting files rather than describing every source file individually.

Runtime Package

- Runtime/Scripts/SimCore - simulation clock, stepping, global APIs, scene configuration, authority, catalog, and RPC.
- Runtime/Scripts/DroneCore - drone lifecycle, per-drone API, commands, controllers, allocation, actuation, cameras, and debug tools.
- Runtime/Scripts/RobotCore - shared robot state, sensor scheduling, sensor models, georeferencing, geomagnetics, and noise.
- Runtime/Scripts/Yaml - YAML discovery, parsing, and typed drone configuration structures.
- Runtime/Scripts/MathUtil - frame conversions and rotor calculations.
- Runtime/Scripts/RuntimeUI - ImGui bootstrap, controls, state displays, settings, and command state.
- Runtime/Scripts/RL - agent state, episode-related scene objects, and scenario management.
- Runtime/Scripts/Experimental - payload attachment, wind volume, following, and trajectory visualization prototypes.
- Runtime/Resources/QuadSim/Configs - packaged default vehicle profiles loaded when a project override is absent.
- Runtime/Prefabs and Runtime/Configuration - simulation root, drone catalog, vehicle prefabs, camera/payload composition, environment objects, and default scene configuration.

Development and Validation

- Editor - YAML inspection, missing-script checks, and rotor-geometry editor visualization.
- Samples~ - the starter scene and internal API example, plus PX4 SITL scripts and the external figure-eight test client.
- Tests - editor tests for clocking, protocol/YAML behavior, rotors/allocation, and play-mode tests for the simulation contract.
- Documentation~ - setup, networking, security, PX4, headless operation, runtime contracts, troubleshooting, validation, and release notes.
- ci~ - built-player smoke testing, Unity log checking, and the minimal Unity test host project.

8. How the Major Pieces Connect

The runtime has one simulation-wide lifecycle and one repeated per-drone lifecycle. Configuration selects the environment and vehicle profile. SimulationManager advances time. DroneManager advances each QuadPawn. Each pawn turns one authorized goal into control outputs, physical forces, and a new sensor state. The API, UI, RPC, camera, and RL features observe or command that same lifecycle rather than maintaining parallel implementations.

Recommended Reading Order

1. Start with SimulationManager.cs to understand the authoritative step loop.
2. Read DroneManager.cs and QuadPawn.cs to see how drones enter and participate in that loop.
3. Read FlightCommandProxy.cs, ModeCoordinator.cs, and CascadedController.cs to follow an input goal into a requested wrench.
4. Read ControlAllocator.cs and Quadactuationmodel.cs to follow the wrench into Rigidbody forces and torques.
5. Read SensorManager.cs and Frames.cs to see how post-physics state becomes external sensor data.
6. Read SimCoreApi.cs, QuadSimApi.cs, and ExternalRpcAdapter.cs to see how local and external clients control the same runtime.
7. Use QuadSimEnvironmentBootstrap.cs, DroneConfigLoader.cs, and the YAML profiles to understand how a scene chooses its runtime behavior.

Boundary Summary

- SimCore owns global lifecycle; DroneCore depends on it for stepping but SimCore does not depend on individual drones.
- QuadPawn owns subsystem coordination; controllers and sensors do not independently schedule Unity physics.
- Configuration supplies parameters; it does not replace runtime ownership or API boundaries.
- UI and RPC are adapters around the APIs; they do not contain a second flight-control implementation.
- Tests and examples exercise the same public surfaces expected to be used after release.